Reviewer 1dYR

Review by Reviewer 1dYR

Paper Summary

This paper proposes a reduced-order model for PDEs. It learns a spatial bank and a small nonlinear head, then solves the projected PDE residual at query time. The bank is ordered after training, so users can change its width or choose between the nonlinear head and the linear span to adjust accuracy and speed.

Presentation: 3: Clear and easy to follow.

Informativeness: 2: I learned something interesting or useful.

Soundness: 2: I have minor or fixable concerns about the paper's soundness.

Critical Strengths

The separation of the spatial bank and the head makes linear terms easy to precompute and reduces the online cost.

Choosing different bank widths without retraining is useful. The paper also reports failed settings and limitations clearly.

Critical Weaknesses

PCA ordering, operator precomputation and empirical quadrature are existing techniques. The paper does not directly compare with CROM or CoLoRA, although it identifies neural-field ROMs as its closest relatives. The gain over these methods is therefore unclear.

The neural operator comparison is not well controlled. Appendix B states that Burgers operators use 128 training trajectories, while the proposed bank and head use 576 and 4608. The operators also have a 3000-second training limit; the heat FNO only completes 6-7 epochs at high resolutions (Table B.3). These differences make the accuracy comparison difficult to interpret.

The reported speedups depend on the selected full-order solvers. Poisson and three-dimensional heat use unpreconditioned CG comparators. Also, Table 1 compares both model settings with a solver chosen to match the accurate setting. The fast setting therefore needs a separate comparison at a similar error level.

Most headline results use the linear span. The nonlinear head works well on Navier-Stokes, but its bank is a POD basis, and both bank and head are built separately for each mesh (Appendix A.5). These results do not clearly establish the combined benefit of the nonlinear head and the mesh-independent coordinate bank.

Questions For Authors

Could the authors compare the neural operators using the same training trajectories and allow enough training to check whether the accuracy gap remains?

How much speedup remains against a stronger full-order solver when each model setting is compared at a similar error level?

Could the authors compare with a neural-field ROM and separate the gains from the coordinate bank, nonlinear head and hyper-reduction?

Flag For Ethics Review: No

Rating: 2: Weak rejection

Confidence: 3: I am familiar with this topic and its relevant literature.

Reply to Reviewer 1dYR

Thank you for your review. We have added the comparisons and ablations you requested and address the concerns and questions you raised below.

W1. Contribution and neural-field baselines

Our contribution is the bank/head factorisation: exact precomputed linear terms, a nonlinear constraint on bank coefficients, and nested widths from one training run. PCA ordering and quadrature are established techniques. We now compare with same-data re-implementations of CROM and CoLoRA (Table 4; Appendix F.1). Our head has 4–5.5× lower worst-case error and 16–87× faster queries than CROM, although CROM's accurate setting has lower median error. CoLoRA has lower worst-case error; our span has lower median error at slightly lower cost. CoLoRA receives initial-condition parameters; ours receives a field.

W2. Matched training data and budgets

We retrained FNO and U-Net on the same 4,608 trajectories, allowing up to 8.1 hours. We compare prediction errors here: our Burgers median error is 7–9× lower, while operators have lower worst-case error (Table 4; Appendix F.2). This is an accuracy-only comparison with direct prediction models; our online method solves the projected PDE. Most runs reached their time limit before convergence, so the results are budget-limited.

W3. Stronger, accuracy-matched solvers
We now match each setting to its own error. The fast width-192 head reaches 0.35–0.48 % worst-case error at 5.1–10.2× over the fastest equal-accuracy pseudo-spectral Navier–Stokes solver from 450 configurations (Table 1; Appendix E.1). Errors cover three models and 128 fresh cases; speedups use the two seeds passing timing checks. Section 5.3 and Appendix C.3 also disclose transform-solver wins on box Poisson/heat 3D and the original Navier–Stokes family's lack of consistent advantage after mesh coarsening.
W4. Coordinate bank and nonlinear head
Table 1 and Figure 2 use per-mesh POD banks. We separately test one coordinate-network bank across meshes: it matches POD at 64^3/96^3 but trails at 128^3 (Table E.1). The earlier cohort and fallback selection after missing the preset rank-64 transfer tolerance are disclosed. Interpolated POD transfers equally well on periodic boxes, so we remove the claim that POD must be rebuilt per mesh.
Q1. Does the operator accuracy gap remain with matched training?
The accuracy result is mixed, as reported under W2: our median error is lower, while the operators have lower worst-case error. Further operator training is running; we will report the updated error comparison by tomorrow (14 November).
Q2. How much speedup survives stronger solvers?
The new Navier–Stokes result is given under W3. Table 3 and Appendix C.3 report the corrected original comparisons, including cases where stronger solvers win.
Q3. What does each component contribute?
We added the neural-field comparisons in W1 and component ablations in Table F.3. At 16 Burgers coordinates, the head gives 2.56 % error versus 56.9 % for a linear closure and 31.8 % for a quadratic one. Sparse nonlinear evaluation reduces time at unchanged error. Appendix F.3 separates these gains from the coordinate bank's mesh-transfer role.

Reviewer iMp8

Review by Reviewer iMp8

Paper Summary

The paper proposes a reduced-order model for PDEs. The solution is a sum of R basis functions (small coordinate networks times a boundary factor) with coefficients from a small head network. After training, the basis is reordered by PCA so that the first R′ functions give a cheaper and less accurate model. At query time, the coefficients are found by minimizing the PDE residual, using precomputed linear terms and empirical quadrature. Tests on Poisson,

heat, Burgers and 3D Navier–Stokes report speedups up to 27.5× (nonlinear) and over 100× (linear).

Presentation: 3: Clear and easy to follow.

Informativeness: 1: I learned little or nothing new from this paper.

Soundness: 1: I have significant concerns about the paper's soundness that do not appear easy to address.

Critical Strengths

Clear cost model: space and latent code are separated, so linear terms can be precomputed.

Timing setup is described in detail, with named baselines.

Limitations are partly acknowledged.

Critical Weaknesses

Weak motivation. The test problems (Poisson, heat, Burgers on a square or cube) are cheap to solve. Modern GPU solvers (FFT, multigrid, GPU finite elements) solve them in milliseconds to seconds with batch (cudss) and can generate thousands of samples in seconds to minutes. There is no point in solving these benchmarks and stressing speed nowadays.

Limited novelty. All parts are known. Neural-field ROMs, hard Dirichlet enforcement.

Mesh independence is overstated. For Navier–Stokes, the basis and head are rebuilt for each mesh. For other problems, there is still a per-mesh setup cost (up to 869 s).

Small and weak evaluation. Test sets are small (6–64 cases), and timings have no spread.

Poor presentation. The main text has one figure (a pipeline diagram). There are no solution plots, error maps, failure cases, or learned basis plots.

Questions For Authors

What is the total cost (data, training, setup), and how many queries are needed to break even?

How is the basis loss defined, and are coefficients solved exactly at each step?

Can it handle other boundary conditions and complex geometry without a hand-made boundary factor?

Please compare with FFT/multigrid GPU solvers and show solution visualizations.

Flag For Ethics Review: No

Rating: 1: Clear rejection

Confidence: 2: I have broad familiarity with this field but little or no direct research experience on this topic.

Reply to Reviewer iMp8

Thank you for your review. We have strengthened the solver comparisons, clarified the scope and cost of the method, and added the requested visual evidence.

W1. Motivation and comparisons with modern solvers

We disagree that fast GPU solvers remove the motivation for reduced models. Our target is repeated PDE queries after offline training and assembly, with query-time accuracy–cost control. We compare against accuracy-matched GPU solvers and distinguish reduced-solve cost from full-query cost.

Our Navier–Stokes head is 3.3–11.7× faster than the fastest accuracy-matched solver from a 450-entry GPU pseudo-spectral ladder; the fast width-192 setting gives 5.1–10.2× (Table 1; Appendix E.1). Incompressibility already uses modewise projection, so there is no sparse pressure system for multigrid or cuDSS to accelerate. Batching both codes gives 17× at 128^3 and 75× at 64^3 for batch 128, including our input/output work.

Figure G.3 demonstrates the scaling advantage on a GPU: increasing the Burgers 2D resolution from 256^2 to 4096^2, a 256-fold increase in grid points, leaves reduced-solve time essentially unchanged at fixed bank width and solver settings on one H200. Input projection and full-field output remain mesh-dependent; our flat-cost claim concerns the reduced solve.

We retain the measured limits: transforms win on box Poisson/3D heat, and input/output cost removes a consistent full-query advantage at 256^3. We are running GPU cuDSS and AmgX single-solve benchmarks and will report the results as soon as they are available, separately from complete PDE-query comparisons.

W2. Contribution

The contribution is a physics-based reduced model with a query-time accuracy–cost ladder from one training run. Bank/head factorisation makes linear terms exact precomputed matrices and constrains online dynamics within the bank span. Figure 2 and Table F.3 isolate the head's benefit; Table 4 compares neural-field baselines. Sections 3 and 5.5 distinguish the contribution from established PCA ordering and quadrature.

W3. Mesh independence and setup

Mesh independence concerns the learned representation and reduced online computation. Per-mesh operator assembly is an explicit offline cost. Table E.1 demonstrates one coordinate bank across meshes, including unchanged-head reuse. Table 1 uses per-mesh POD and is labelled accordingly. Interpolated POD also transfers on periodic boxes; our claim does not require rebuilding POD on every mesh.

W4. Evaluation size and timing variability
Our main revised evaluation uses 128 fresh cases and three independently trained models (Table 1). Appendix C.4 reports GPU timing spread. Speedups use the two seeds passing timing checks. The third seed's accuracy remains included, with its failed timing gate disclosed in Appendix E.1. Exploratory cohorts and failed retrains are labelled.
W5. Solution plots and presentation
We added a main-text head-versus-span comparison (Figure 2). Appendix G supplies solution fields, error maps, learned bases, failure cases, mesh scaling and break-even curves.
Q1. Total cost and break-even
We target repeated queries within one parameterised family. Appendix C.2, Table C.4 reports offline costs and break-even counts: about 43,000 queries for Navier–Stokes at 128^3. Missing data-generation costs are identified. Against transforms, box Poisson never breaks even, and heat 2D needs about 370,000 queries.
Q2. Basis loss and exact coefficient solves
We give the losses and weights in Appendix C.1, Eqs. (20)–(23) and Table C.3. Only heat 3D and Burgers 3D eliminate coefficients at every training step through thin QR. Heat 3D uses a weighted POD-mode mean and a snapshot-minibatch tail term. Other banks jointly train coefficients or latent codes, with the stated least-squares resets/phases. After freezing the bank, ordering coefficients and head targets are exact least-squares fits; online coefficients come from the projected PDE solve.
Q3. Geometry and boundary conditions
We demonstrate cylinder flow using smooth boundary-distance factors and an inflow lift in a Galerkin rollout (Appendix E.2). Automatic handling of arbitrary geometry and Neumann/Robin conditions remains untested.
Q4. FFT/multigrid comparisons and visualisations

See W1 for the GPU numerical-solver comparisons and W5 for visualisations. We are running GPU cuDSS and AmgX single-solve benchmarks and will report the results as soon as they are available.

Reviewer 8ZEF

Review by Reviewer 8ZEF

Paper Summary

This paper proposes a runtime-tunable reduced-order modeling framework for efficient PDE solving. It combines a learned coordinate-based spatial bank with linear-span and nonlinear-manifold representations, using matrix-free Petrov–Galerkin projection for efficient online solving. The ordered spatial bank allows users to adjust the accuracy–efficiency trade-off without retraining. Experiments on Poisson, heat, Burgers, and Navier–Stokes equations demonstrate promising accuracy and speedups over full-order solvers and existing learning-based approaches.

Presentation: 2: Generally understandable, but requires substantial effort in places.

Informativeness: 2: I learned something interesting or useful.

Soundness: 3: I have no substantive concerns about the paper's soundness.

Critical Strengths

Practical and flexible design. The ability to switch between linear-span and nonlinear-manifold representations and adjust the basis width at inference time is useful. The integration of coordinate-based spatial banks, precomputation, and hyper-reduction makes the framework computationally efficient.

Strong empirical results. The experiments cover multiple PDE families and resolutions, demonstrating substantial speedups and encouraging cross-resolution performance. The comparisons with both conventional ROMs and neural operators provide useful evidence of the method's practical value.

Critical Weaknesses

Limited diversity of nonlinear evaluations. Although the paper includes Burgers and Navier–Stokes equations, the most accurate configurations for several tasks use the linear-span representation. Additional challenging nonlinear PDE benchmarks would help establish when the nonlinear-manifold component provides substantial benefits and how broadly the framework generalizes.

Some limitations in baseline comparisons. The reported speedups depend on the selected full-order solvers, and the neural operator comparisons use different training data sizes and computational budgets in some settings. More closely matched comparisons would strengthen the empirical conclusions.

Questions For Authors

Could the authors provide additional experiments on more challenging nonlinear PDEs, particularly problems with stronger nonlinear dynamics, to further demonstrate the benefits of the nonlinear-manifold representation?

Could the authors further clarify when the nonlinear head is preferable to the linear span, and whether its advantages extend beyond the current Navier–Stokes setting?

How might the reported speedups change when compared with more optimized full-order solvers or neural operators trained under more closely matched budgets?

Minor Suggestion: The paper is generally well organized and easy to follow. However, Figure 1 could be more visually polished, with a cleaner layout, more consistent typography, and clearer separation between offline training and online inference. This would further improve the presentation.

Flag For Ethics Review: No

Rating: 3: Weak acceptance

Confidence: 3: I am familiar with this topic and its relevant literature.

Reply to Reviewer 8ZEF

Thank you for your constructive review. We have added nonlinear benchmarks, matched comparisons and clearer figures in response to your suggestions.

W1. Diversity of nonlinear evaluations

We added interacting-vortex Navier–Stokes with sharper initial data and lower viscosity (§5.1, Table 1), and unsteady cylinder flow (§5.2, Table 2). The fast Navier–Stokes head stays below 0.5 % worst-case error at 5.1–10.2× over its matched solver, while the same bank's linear span fails. The cylinder demonstrates non-box geometry and query-time width choices; it uses linear-span Galerkin and does not establish an additional nonlinear-head advantage.

W2. Fairness of the comparisons

We separate the comparisons: numerical solvers are matched to each setting's accuracy (§5.3), while neural operators are compared by prediction error under the reported training budgets. With the same 4,608 Burgers trajectories, our median error is 7–9× lower; operators have lower worst-case error (Table 4; Appendix F.2). The budgets do not establish convergence. Transform solvers win on box Poisson/3D heat.

Q1. Additional challenging nonlinear experiments

The added benchmarks are described under W1. Cylinder widths 24/32 retain all six targets over a window three times longer; extrapolation outside the training band remains limited (Table E.2). We state these limits alongside the results.

Q2. When is the nonlinear head preferable?

The head helps when few unknowns are affordable and a linear span fails (Figure 2). At 16 Burgers coordinates, errors are 2.56 % for the head, 56.9 % for a linear closure and 31.8 % for a quadratic one (Table F.3). A wider span can be faster; §5.5 explains this choice.

Q3. Stronger solvers and matched neural operators

W2 summarises the corrected comparison. For cylinder flow, FNO meets all targets on the standard window, but no tested operator does on the longer window or outside the training band (Table F.2). Further operator error comparisons are running; we will report them by tomorrow (14 November).

Minor suggestion. Figure 1

We redrew Figure 1 to separate offline training and online inference, with a cleaner layout and consistent typography.

General response

Part 1 of 2

We thank the reviewers. We added more demanding nonlinear problems, accuracy-matched GPU numerical solvers and matched-data operator error comparisons. The key results follow; references are to the revised manuscript. We are running GPU cuDSS and AmgX single-solve benchmarks and will report the results as soon as they are available. Further operator-accuracy results will be reported by tomorrow (14 November).

Table G1 (revised Table 1; §5.1, Appendix E.1). Navier–Stokes with sharper vortices and lower viscosity: 128 fresh cases, three independently trained models, per-mesh POD banks. Each setting faces the fastest equally accurate configuration from a 450-entry solver ladder.

Head on per-mesh POD width	Worst-case error, three seeds (64^3 / 96^3 / 128^3)	Median error	Time (64^3 / 96^3 / 128^3)	Matched speedup
64	1.17–1.36 / 1.45–1.61 / 1.43–1.69 %	0.34–0.54 %	16 / 17 / 31 ms	6.9 / 6.4–6.5 / 3.6–3.7×
96	0.58–0.75 / 0.64–0.78 / 0.62–0.82 %	0.17–0.19 %	17 / 26 / 43 ms	9.4–11.7 / 7.6–9.5 / 4.7–6.9×
128	0.42–0.60 / 0.43–0.62 / 0.45–0.64 %	0.10–0.12 %	31 / 48 / 62 ms	6.3–6.5 / 5.2–7.0 / 4.0–5.3×

128, fast	0.49–0.67 / 0.52–0.68 / 0.47–0.69 %	0.12–0.14 %	19 / 31 / 51 ms	9.4–10.4 / 8.0–10.9 / 4.9–6.1×
192	0.28–0.39 / 0.35–0.41 / 0.35–0.42 %	0.06–0.08 %	52 / 67 / 121 ms	3.9–5.9 / 5.9–6.2 / 3.3–3.4×
192, fast	0.35–0.48 / 0.39–0.47 / 0.43–0.47 %	0.09–0.11 %	25 / 40 / 68 ms	8.0–10.2 / 8.3–10.0 / 5.1×
Linear span, same banks	7.7–70 %	—	—	0.04–0.56×

The fast width-192 head stays below 0.5 % on every mesh/seed at 5.1–10.2×. Errors cover three seeds; speedups use the two passing timing checks. Times are solve-only. Appendix E.1 and Table C.7 give selection, timing spread and the failed timing gate.

A separate, earlier cohort tests one coordinate network across meshes (Table E.1): it matches POD at 64^3/96^3 but trails at 128^3. The table reports fallback-selection caveats and unchanged-head reuse; interpolated POD transfers equally well.

Table G3 (Figure G.3). Burgers 2D reduced-solve cost at fixed width, using a later off-mesh solver with the frozen bank; six development cases and three repeats on one H200. Output reconstruction is charged separately.

Width	**Evolution, 256^2 / 1024^2 / 4096^2**	**Mean iterations**	**Six-field reconstruction**
128	22.1 / 22.0 / 21.1 ms	51.8 / 51.7 / 51.7	0.13 / 0.40 / 5.7 ms
256	51.7 / 51.6 / 51.8 ms	53.8 / 54.2 / 54.2	0.15 / 0.67 / 10.0 ms
384	87.8 / 86.6 / 88.3 ms	54.3 / 54.3 / 54.3	0.18 / 0.95 / 14.3 ms

Reduced evolution is flat across these meshes; this variant remains slower than the best matched solver at 4096^2 (0.05–0.71×). Navier–Stokes likewise has flat reduced-solve cost, but input/output work grows; its full query has no consistent speed advantage over the matched solver at 256^3 (Appendix E.1).

Part 2 of 2

Table G2 (revised Table 2; §5.2, Appendix E.2). Cylinder accuracy: model on a P2/P1 mesh with 96,781 unknowns; truth from a separate P4/P3 reference at step 1/3200. Train Re 90/100/110, development 95, held-out 105.

One trained bank	Velocity error
Cheapest passing finite-element run	9.4e-3
Width 32, step 1/400	7.0e-3
Width 24, step 1/400	5.5e-3
Width 16, step 1/800	8.3e-3

Galerkin with an inflow lift, matched in accuracy by equal-width POD. Widths 24/32 retain all six targets over a three-times-longer window; out-of-band Re 80/120 remain failures (Table E.2).
Table G4 (Appendix E.1). Batched Navier–Stokes: fast width-192 head versus its matched pseudo-spectral solver, both batched on the same GPU; batched outputs reproduce the single-query code. Ratio = solver time / our time per query.

Mesh	1 query	8	32	128	Time per 1,000 queries at batch 128 (ours / solver)
64^3	9.4×	39.8×	64.5×	75.1×	1.6–2.1 s / 117–159 s
96^3	11.6×	35.7×	46.7×	48.4×	—
128^3	6.5×	15.4×	17.3×	17.1×	about 11 s / about 3 min

Our timings include input projection and all output fields. The 96^3, batch-128 cell uses an H200; other cells use an A100. Each ratio compares codes within one job, separate from G1.
Corrections and scope. Section 5.3 and Appendix C.3 give the corrected original comparisons: own-error matching, stronger solvers and transform-solver wins. Table 4 and Appendix F disclose the mixed neural-field/operator results and budget limits; PCA ordering and quadrature are established techniques. We retain the implementation corrections in Appendices A/C.
The supported claim is query-time accuracy–cost choice from one trained model, with a head that helps when small linear spans fail. Table 1 demonstrates speedups over the tested accuracy-matched GPU solver on the Navier–Stokes family; it does not establish universal superiority or mesh-flat full-query cost.

Draft confidential note to the Area Chair

For a separate AC-only comment; not part of the public reviewer replies.

Dear Area Chair,

We would like to distinguish the broad motivation question raised in reviewer iMp8's W1 from the paper-specific methodological and empirical assessment. The availability of fast full-order solvers is a reason to require strong comparisons; it does not by itself establish that reduced models have no useful role.

Our contribution is a learned low-dimensional representation with query-time accuracy–cost choices, while retaining a projected PDE solve for each query. The intended use is repeated simulation within a parameterised family. The revised manuscript addresses the specific baseline and evaluation concerns through accuracy-matched numerical solvers, matched-data operator error comparisons, fresh test cases and independently trained models.

In particular, Table 1 provides measured advantages over the tested accuracy-matched GPU solver on the stated Navier–Stokes family. We also report transform-solver wins, offline amortisation costs and the limits of mesh-independent reduced computation. We respectfully ask that the assessment distinguish the general motivation question from this narrower contribution and revised evidence.